ASSIGNMENT – 13.5

2303A51655
Batch : 29

Task Description #1 (Refactoring – Removing Global Variables)

- **Task: Use AI to eliminate unnecessary global variables from the code.**
- **Instructions:**

o **Identify global variables used across functions.**
o **Refactor the code to pass values using function parameters.**
o **Improve modularity and testability.**

- **Sample Legacy Code:**

rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))

- **Expected Output:**

o **Refactored version passing rate as a parameter or using a configuration structure.**

```
task1.py > ...
def calculate_interest(amount, rate):
        rate (float): Interest rate

    Returns:
        float: Calculated interest
    """
    return amount * rate


rate = 0.1
print(calculate_interest(1000, rate))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.p
y/assignment - 13.5.py/task1.py"
100.0
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation:

- Global variables reduce modularity and make code harder to test.
- Passing variables as function parameters improves code flexibility.
- Refactoring removes hidden dependencies between functions.
- Modular code improves readability and maintainability.
- AI assistance helps identify global variables and refactor them efficiently.

Task Description #2 : (Refactoring Deeply Nested Conditionals)
• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o **Readability**
o **Logical simplification**
o **Maintainability**
Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:
o **Flattened logic using guard clauses or a mapping-based approach.**

task2.py > ...

```python
def evaluate_score(score):
    """
    Evaluate performance based on score.
    """
    if score >= 90:
        return "Excellent"
    if score >= 75:
        return "Very Good"
    if score >= 60:
        return "Good"
    return "Needs Improvement"


score = 78
print(evaluate_score(score))
```

Output:

```
● arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent
  y/assignment - 13.5.py/task2.py"
  Very Good
✧ arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation:

- Deeply nested conditional statements reduce code readability.
- Flattening logic using guard clauses simplifies the structure.
- Refactored code becomes easier to understand and maintain.
- Logical simplification reduces chances of programming errors.
- AI helps restructure complex conditional logic into cleaner patterns.

Task 3 (Refactoring Repeated File Handling Code)
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse
Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:
o Reusable function using with open() and parameters.

```python
task3.py > ...

# generate a Python program for a Smart Healthcare Appointment Scheduling System.

# System contains appointment records with:
# appointment_id, patient_name, doctor_name, appointment_time, consultation_fee

# Requirements:
# 1. Recommend suitable algorithms for:
#    - Searching appointment using appointment_id: Linear search, as appointment IDs may not be
#    - Sorting appointments by time: Timsort (Python's built-in sorted), efficient for general s
#    - Sorting appointments by consultation_fee: Timsort, suitable for numerical sorting.
# 2. Justify the chosen algorithms in 2-3 sentences.
#    Linear search is simple and works for unsorted data with small datasets. Timsort is stable and efficient,
# 3. Implement the searching algorithm in Python.
# 4. Implement the sorting algorithm in Python.
# 5. Use clear functions such as:
#    search_appointment()
#    sort_by_time()
#    sort_by_fee()
# 6. Add docstrings and detailed comments.
# 7. Provide sample appointment data to test the program.

# Sample appointment data

appointments = [
    {"appointment_id": 1, "patient_name": "Alice", "doctor_name": "Dr. Smith", "appointment_time": "2024-07-01
    {"appointment_id": 2, "patient_name": "Bob", "doctor_name": "Dr. Johnson", "appointment_time": "2024-07-01
    {"appointment_id": 3, "patient_name": "Charlie", "doctor_name": "Dr. Smith", "appointment_time": "2024-07-0
    {"appointment_id": 4, "patient_name": "David", "doctor_name": "Dr. Johnson", "appointment_time": "2024-07-0
    {"appointment_id": 5, "patient_name": "Eve", "doctor_name": "Dr. Smith", "appointment_time": "2024-07-01 08
]

def search_appointment(appointments, appointment_id):
    """
    Searches for an appointment by appointment_id using linear search.
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task3.py"
1 08:00', 'consultation_fee': 110}
{'appointment_id': 3, 'patient_name': 'Charlie', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-
07-01 09:00', 'consultation_fee': 120}
{'appointment_id': 4, 'patient_name': 'David', 'doctor_name': 'Dr. Johnson', 'appointment_time': '2024-
07-01 12:00', 'consultation_fee': 130}
{'appointment_id': 2, 'patient_name': 'Bob', 'doctor_name': 'Dr. Johnson', 'appointment_time': '2024-07
-01 11:00', 'consultation_fee': 150}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

- Repeated file operations violate the **DRY (Don't Repeat Yourself)** principle.

- Using reusable functions reduces code duplication.

- Context managers like with open() automatically handle file closing.
- Refactoring improves code reliability and readability.
- AI assistance helps identify repeated patterns and optimize them.

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o **Time complexity**

o **Data structure choice**

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o **Use of sets or dictionaries with complexity justification.**

```python
users = {"admin", "guest", "editor", "viewer"}

def check_access(name):
    """
    Check if user exists in the system.
    """
    return "Access Granted" if name in users else "Access Denied"


name = input("Enter username: ")
print(check_access(name))
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 13.5.py/task4.py"
Enter username: user
Access Denied
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation:

- Linear search becomes inefficient for large datasets.
- Using sets or dictionaries improves search efficiency.
- Optimized data structures reduce time complexity significantly.
- Refactoring improves overall system performance.
- AI helps recommend appropriate data structures for faster searching.

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

```python
class EmployeeSalaryCalculator:
    """
    Class to calculate employee net salary.
    """

    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def net_salary(self):
        return self.salary - self.calculate_tax()


emp = EmployeeSalaryCalculator(50000)
print(emp.net_salary())
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 13.5.py/task5.py"
40000.0
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation:

1. Procedural code can become difficult to manage as programs grow.
2. Object-Oriented Programming organizes code into classes and objects.
3. Encapsulation improves data security and structure.
4. Class-based design enhances scalability and maintainability.
5. AI assistance helps convert procedural logic into OOP-based designs.

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or

comprehensions, with time comparison.

task6.py > ...

```python
def sum_even_numbers(n):
    """
    Calculate sum of even numbers up to n.
    """
    return sum(i for i in range(1, n) if i % 2 == 0)


print(sum_even_numbers(1000000))
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.p
y/assignment - 13.5.py/task6.py"
249999500000
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation :

1. Large loops can reduce program performance significantly.
2. Mathematical formulas or built-in functions provide faster solutions.
3. Algorithmic optimization reduces time complexity.
4. Refactoring improves program efficiency and scalability.
5. AI helps identify performance bottlenecks and suggest optimized logic.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy

Code:

```
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

o Refactored function returning values instead of mutating

globals.

```python
def add_item(data, x):
    """
    Add item and return new list.
    """
    new_data = data.copy()
    new_data.append(x)
    return new_data


data = []
data = add_item(data, 10)
data = add_item(data, 20)

print(data)
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 13.5.py/task7.py"
[10, 20]
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation:

- Modifying global mutable variables creates hidden side effects.
- Returning values from functions improves predictability.
- Functional-style programming reduces unexpected behavior.
- Refactoring improves code clarity and maintainability.
- AI assistance helps detect shared state issues and suggest safer alternatives.

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

o Testability

Legacy

Code:

```
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

task8.py > ...

```python
def is_long(password):
    return len(password) >= 8


def has_digit(password):
    return any(c.isdigit() for c in password)


def has_uppercase(password):
    return any(c.isupper() for c in password)


def validate_password(password):
    """
    Validate password based on multiple rules.
    """
    if not is_long(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"


password = input("Enter password: ")
print(validate_password(password))
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assistent.p
y/assignment - 13.5.py/task8.py"
Enter password: 1234
Password too short
arepallyramcharan@Arepallys-MacBook-Air assignment - 13.5.py %
```

Observation:

1. Complex validation logic becomes difficult to read and maintain.
2. Breaking validation into smaller functions improves clarity.
3. Modular validation makes testing easier.
4. Refactored code improves maintainability and scalability.

5. AI helps simplify and structure validation rules effectively.